main.c

Run ▶

```c
#include<stdio.h>
#include<stdlib.h>

struct Node
{
    int data;
    struct Node *left,*right;
};

struct Node* create(int x)
{
    struct Node *p;
    p=(struct Node*)malloc(sizeof(struct Node));
    p->data=x;
    p->left=p->right=NULL;
    return p;
}

void preorder(struct Node *p)
{
    if(p!=NULL)
    {
        printf("%d ",p->data);
        preorder(p->left);
```

main.c

Run ▶

```
}
void inorder(struct Node *p)
{
    if(p!=NULL)
    {
        inorder(p->left);
        printf("%d ",p->data);
        inorder(p->right);
    }
}
void postorder(struct Node *p)
{
    if(p!=NULL)
    {
        postorder(p->left);
        postorder(p->right);
        printf("%d ",p->data);
    }
}
int main()
{
    struct Node *root;
    root=create(1);
    root->left=create(2);
```

```
    }
}
int main()
{
    struct Node *root;
    root=create(1);
    root->left=create(2);
    root->right=create(3);
    root->left->left=create(4);
    root->left->right=create(5);

    printf("Preorder traversal: ");
    preorder(root);
    printf("\nInorder traversal: ");
    inorder(root);
    printf("\nPostorder traversal: ");
    postorder(root);
    return 0;
}
```

```
#include<stdio.h>
#include<stdlib.h>

struct Node
{
   int data;
   struct Node *left,*right;
};

struct Node* create(int x)
{
   struct Node *p;
   p=(struct Node*)malloc(sizeof(struct Node));
   p->data=x;
   p->left=p->right=NULL;
   return p;
}

void preorder(struct Node *p)
{
   if(p!=NULL)
   {
      printf("%d ",p->data);
      preorder(p->left);
```

```
        preorder(p->right);
    }
}
void inorder(struct Node *p)
{
    if(p!=NULL)
    {
        inorder(p->left);
        printf("%d ",p->data);
        inorder(p->right);
    }
}
void postorder(struct Node *p)
{
    if(p!=NULL)
    {
        postorder(p->left);
        postorder(p->right);
        printf("%d ",p->data);
    }
}
int main()
{
    struct Node *root;
    root=create(1);
    root->left=create(2);
    root->right=create(3);
    root->left->left=create(4);
    root->left->right=create(5);

    printf("Preorder traversal: ");
    preorder(root);
    printf("\nInorder traversal: ");
    inorder(root);
    printf("\nPostorder traversal: ");
    postorder(root);
    return 0;
}
```

Output

```
Preorder traversal: 1 2 4 5 3
Inorder traversal: 4 2 5 1 3
Postorder traversal: 4 5 2 3 1
```